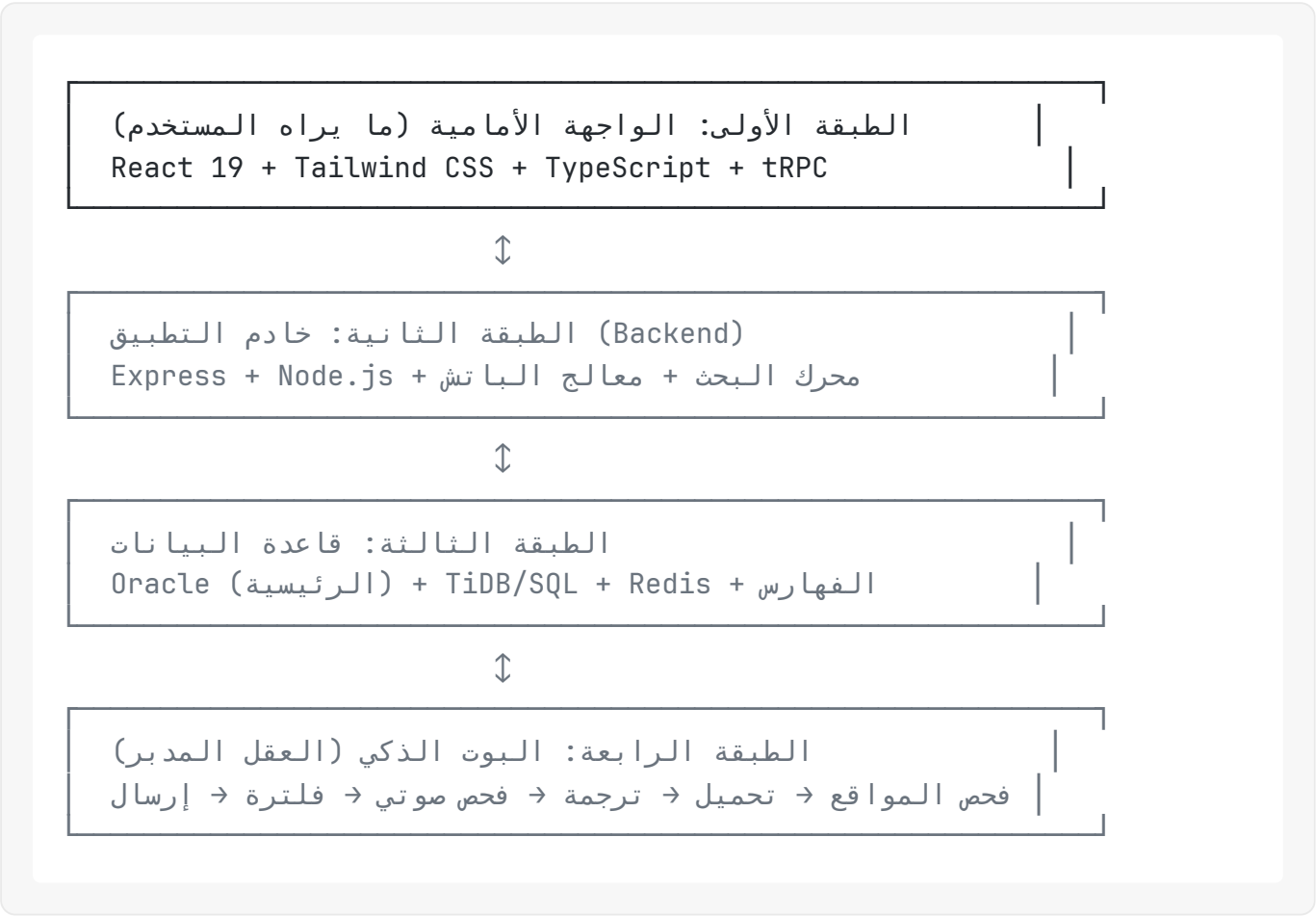


البنية التحتية لمنصة فحص العقوبات

المعمارية الكلية

الرئيسية (الحقل المستضيف) Oracle المنصة مبنية على معمارية متعددة الطبقات حول قاعدة بيانات



الطبقة الأولى: الواجهة الأمامية

لبناء واجهات المستخدم التفاعلية مع دعم الحالة والمكونات JavaScript مكتبة - React 19

حديث لإنشاء تصاميم متجاوبة CSS إطار عمل - Tailwind CSS

لغة برمجة توفر أمان النوع والتحقق من الأخطاء في وقت التطوير - TypeScript

مكتبة لاتصال آمن من الناحية النوعية بين الواجهة الأمامية والخادم - tRPC

مثال من الكود:

```
// client/src/pages/SearchPage.tsx
import React, { useState, useCallback } from 'react';
import { trpc } from '@lib/trpc';

interface SearchResult {
  id: string;
  nameEn: string;
  nameAr: string;
  matchScore: number;
  status: 'match' | 'possible_match' | 'no_match';
}

export const SearchPage: React.FC = () => {
  const [query, setQuery] = useState<string>('');
  const [results, setResults] = useState<SearchResult[]>([]);

  // محمي من الناحية النوعية trpc استدعاء
  const { data, isLoading, error } = trpc.search.sanctionsList.useQuery(
    { query, limit: 50 },
    { enabled: query.length ≥ 2 }
  );

  const handleSearch = useCallback((searchQuery: string) => {
    setQuery(searchQuery);
    if (data) {
      setResults(data);
    }
  }, [data]);

  return (
    <div className="search-container bg-white rounded-lg shadow-lg p-6">
      <input
        type="text"
        value={query}
        onChange={(e) => handleSearch(e.target.value)}
        placeholder="ابحث عن اسم. "
        className="w-full px-4 py-2 border border-gray-300 rounded-lg focus:outline-none"
      />

      {isLoading && <div className="text-center py-4">جاري البحث. </div>}
      {error && <div className="text-red-500 py-4">خطأ في البحث</div>}
```

```

<div className="results-grid grid grid-cols-1 md:grid-cols-2 gap-4"
  {results.map((result) => (
    <div key={result.id} className="result-card bg-gray-50 p-4 rounded
      <h3 className="font-semibold text-lg text-gray-900">{result.name}
      <p className="text-sm text-gray-600">{result.nameAr}</p>
      <p className={`text-sm font-medium mt-2 ${
        result.status === 'match' ? 'text-green-600' :
        result.status === 'possible_match' ? 'text-yellow-600' :
        'text-red-600'
      }`}>
        {result.status === 'match' ? '✓ مطابقة' :
        result.status === 'possible_match' ? '⚠ محتملة' :
        '✗ عدم مطابقة'}
      </p>
      <p className="text-xs text-gray-500 mt-1">درجة التطابق: {result.score}</p>
    </div>
  ))}
</div>
</div>
);
};

```

(Backend) الطبقة الثانية: خادم التطبيق

إطار عمل ويب خفيف الوزن - Express.js

على الخادم JavaScript بيئة تشغيل - Node.js

(Fuzzy Search) محرك البحث - خوارزميات بحث متقدمة

معالج الباتش - معالجة الملفات الكبيرة بكفاءة

مثال من الكود

```
// server/routers.ts - تعريف الإجراءات
import { router, publicProcedure, protectedProcedure } from '@server/_core';
import { z } from 'zod';
import { searchSanctionsList } from '@server/search-engine';

export const appRouter = router({
  search: router({
    sanctionsList: publicProcedure
      .input(z.object({
        query: z.string().min(2).max(100),
        limit: z.number().int().min(1).max(100).optional()
      }))
      .query(async ({ input }) => {
        const results = await searchSanctionsList(input.query);
        return results.slice(0, input.limit || 10);
      })
  }),

  batch: router({
    processBatch: protectedProcedure
      .input(z.object({
        jobId: z.string().uuid()
      }))
      .mutation(async ({ input, ctx }) => {
        // التحقق من الصلاحيات
        if (ctx.user.role !== 'admin' && ctx.user.role !== 'operator') {
          throw new Error('Unauthorized');
        }

        const job = await db.query.batchJobs.findOne({
          where: { id: input.jobId }
        });

        return await processBatchJob(job);
      })
  })
});
```

محرك البحث:

```
// server/search-engine.ts
import Fuse from 'fuse.js';
import { levenshteinDistance } from 'js-levenshtein';

export async function searchSanctionsList(query: string): Promise<SanctionRecord[]> {
  const normalizedQuery = normalizeArabicText(query);
  const queryTokens = tokenize(normalizedQuery);

  // جلب السجلات من قاعدة البيانات
  const allRecords = await db.query.sanctionsRecords.findAll();

  // Fuse.js البحث الضبابي باستخدام
  const fuse = new Fuse(allRecords, {
    keys: ['nameEn', 'nameAr', 'aliases'],
    threshold: 0.4,
    includeScore: true,
    minMatchCharLength: 2
  });

  let results = fuse.search(normalizedQuery);

  // فلترة بناءً على عدد الكلمات المتطابقة (3 كلمات على الأقل)
  results = results.filter(result => {
    const matchedWords = countMatchedWords(queryTokens, result.item);
    return matchedWords ≥ 3;
  });

  return results.map(result => ({
    ...result.item,
    matchScore: result.score ? (1 - result.score) * 100 : 0,
    status: getMatchStatus(result.score)
  }));
}

function countMatchedWords(queryTokens: string[], record: SanctionRecord) {
  const recordTokens = tokenize(
    `${record.nameEn} ${record.nameAr} ${record.aliases?.join(' ') || ''}`
  );

  let matchedCount = 0;
  for (const token of queryTokens) {
    const recordIndex = recordTokens.indexOf(token);
    if (recordIndex !== -1) {
      matchedCount++;
    }
  }
}
```

```
for (const recordToken of recordTokens) {  
    const distance = levenshteinDistance(token, recordToken);  
    const similarity = 1 - (distance / Math.max(token.length, recordToken.length));  
  
    if (similarity ≥ 0.8) {  
        matchedCount++;  
        break;  
    }  
}  
}  
  
return matchedCount;  
}
```

الطبقة الثالثة: قاعدة البيانات

القاعدة الرئيسية (الحقل المستضيف) - Oracle Database

نسخة موازية لتوزيع الحمل - TiDB/MySQL

تخزين مؤقت سريع - Redis

الفهارس المتقدمة - تحسين أداء الاستعلامات

مثال من الكود

```
// server/db.ts
import { drizzle } from 'drizzle-orm/mysql2';
import { createPool } from 'mysql2/promise';
import Redis from 'redis';

// اتصال قاعدة البيانات الرئيسية (Oracle)
const oraclePool = createPool({
  host: process.env.ORACLE_HOST,
  user: process.env.ORACLE_USER,
  password: process.env.ORACLE_PASSWORD,
  database: process.env.ORACLE_DB,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

// للتخزين المؤقت Redis
const redisClient = Redis.createClient({
  host: process.env.REDIS_HOST,
  port: parseInt(process.env.REDIS_PORT || '6379'),
  password: process.env.REDIS_PASSWORD
});

export const db = {
  async searchSanctions(query: string) {
    const cacheKey = `search:${query}`;

    // التحقق من الكاش أولاً
    const cached = await redisClient.get(cacheKey);
    if (cached) {
      return JSON.parse(cached);
    }

    // البحث في قاعدة البيانات
    const connection = await oraclePool.getConnection();
    try {
      const [rows] = await connection.query(
        `SELECT * FROM sanctions_records
        WHERE MATCH(name_en, name_ar) AGAINST(? IN BOOLEAN MODE)
        LIMIT 100`,
        [query]
      );
    }
  }
};
```

```

);

// حفظ في الكاش لمدة 1 ساعة
await redisClient.setEx(cacheKey, 3600, JSON.stringify(rows));

return rows;
} finally {
    connection.release();
}
},

async insertSanctionRecord(record: SanctionRecord) {
    const connection = await oraclePool.getConnection();
    try {
        await connection.beginTransaction();

        // الإدراج في قاعدة البيانات الرئيسية
        const [result] = await connection.query(
            `INSERT INTO sanctions_records
            (id, name_en, name_ar, entity_type, nationality, created_at)
            VALUES (?, ?, ?, ?, ?, NOW())`,
            [record.id, record.nameEn, record.nameAr, record.entityType, record.nationality, record.created_at]
        );

        // النسخ المرآة في TiDB
        const tidbConnection = await tidbPool.getConnection();
        try {
            await tidbConnection.query(
                `INSERT INTO sanctions_records
                (id, name_en, name_ar, entity_type, nationality, created_at)
                VALUES (?, ?, ?, ?, ?, NOW())`,
                [record.id, record.nameEn, record.nameAr, record.entityType, record.nationality, record.created_at]
            );
        } finally {
            tidbConnection.release();
        }

        await connection.commit();
        await redisClient.del('search:*');

        return result;
    } catch (error) {
        await connection.rollback();
        throw error;
    }
}

```



```

    } finally {
        connection.release();
    }
}
};

// الفهارس المتقدمة
export const createIndexes = async () => {
    const connection = await oraclePool.getConnection();
    try {
        await connection.query(
            `CREATE INDEX idx_name_en ON sanctions_records(name_en) USING HASH`
        );

        await connection.query(
            `CREATE INDEX idx_name_ar ON sanctions_records(name_ar) USING HASH`
        );

        await connection.query(
            `CREATE INDEX idx_entity_nationality
            ON sanctions_records(entity_type, nationality)`
        );

        await connection.query(
            `CREATE FULLTEXT INDEX idx_fulltext
            ON sanctions_records(name_en, name_ar, aliases)`
        );

        console.log('تم إنشاء الفهارس بنجاح ✓');
    } finally {
        connection.release();
    }
};

```

الطبقة الرابعة: البوت الذكي (العقل المدبر للنظام)

البوت يقوم بـ

1. فحص المواقع الرسمية (UNSC, EU, OFAC)
2. تحميل التحديثات تلقائياً
3. ترجمة الأسماء والكيانات من الإنجليزية إلى العربية

4. فحص الترجمة الصوتية والنطقية للتحقق من الدقة.
5. فلترة البيانات حسب الأعمدة المطلوبة.
6. إرسال التحديثات إلى النظام للمراجعة اليدوية.

مثال من الكود:

```
// server/smart-bot.ts
import axios from 'axios';
import { GoogleTranslate } from '@google-cloud/translate';

export class SmartBot {
  private translator = new GoogleTranslate();

  async runDailyUpdate() {
    console.log('البوت: بدء التحديث اليومي 🤖');

    // 1. فحص المواقع الرسمية
    const unscData = await this.fetchUNSCList();
    const euData = await this.fetchEUList();
    const ofacData = await this.fetchOFACList();

    const allData = [...unscData, ...euData, ...ofacData];
    console.log(`✓ سجل تم جلب ${allData.length}`);

    // 2. ترجمة والتحقق
    const translatedRecords = await this.translateAndValidate(allData);
    console.log(`✓ تمت ترجمة ${translatedRecords.length} سجل`);

    // 3. فلترة البيانات
    const filteredRecords = this.filterData(translatedRecords);
    console.log(`✓ تم فلترة ${filteredRecords.length} سجل`);

    // 4. إرسال للمراجعة اليدوية
    await this.sendForReview(filteredRecords);
    console.log(`✓ سجل للمراجعة ${filteredRecords.length} تم إرسال`);
  }

  private async fetchUNSCList() {
    try {
      const response = await axios.get(
        'https://www.un.org/sc/suborg/en/sanctions/un-sc-consolidated-list'
      );
      return parseUNSCHtml(response.data);
    } catch (error) {
      console.error('✗ خطأ UNSC: ', error);
      return [];
    }
  }
}
```

```

}

private async translateAndValidate(records: any[]) {
  const translated = [];

  for (const record of records) {
    try {
      // ترجمة الاسم
      const [translation] = await this.translator.translate(record.nameEn, {
        targetLanguage: 'ar'
      });

      // فحص النطق الصوتي
      const phoneticValidation = this.validatePhonetic(record.nameEn, translation);

      // فحص الترجمة الصوتية
      const audioValidation = await this.validateAudioTranscription(record.nameEn, translation);

      translated.push({
        ...record,
        nameAr: translation,
        phoneticValidation,
        audioValidation,
        translatedAt: new Date()
      });

    } catch (error) {
      console.error(`❌ خطأ في ترجمة ${record.nameEn}:`, error);
    }
  }

  return translated;
}

private filterData(records: any[]) {
  return records.filter(record => {
    return (
      record.nameEn &&
      record.nameAr &&
      record.entityType &&
      record.nationality &&
      record.phoneticValidation &&
      record.audioValidation
    );
  });
}

```

```

    }).map(record => ({
      id: generateUUID(),
      nameEn: record.nameEn.trim(),
      nameAr: record.nameAr.trim(),
      entityType: record.entityType,
      nationality: record.nationality,
      listingReason: record.listingReason || '',
      actionTaken: record.actionTaken || '',
      source: record.source,
      createdAt: new Date(),
      status: 'pending_review'
    }));
  }

  private async sendForReview(records: any[]) {
    await axios.post(
      'http://localhost:3000/api/bot/submit-updates',
      {
        records,
        submittedAt: new Date(),
        botVersion: '1.0.0'
      },
      {
        headers: {
          'Authorization': `Bearer ${process.env.BOT_API_KEY}`,
          'Content-Type': 'application/json'
        }
      }
    );
  }

  private validatePhonetic(englishName: string, arabicName: string): boolean {
    const englishPhonetics = this.getPhonetics(englishName);
    const arabicPhonetics = this.getPhonetics(arabicName);
    const similarity = this.calculateSimilarity(englishPhonetics, arabicPhonetics);
    return similarity > 0.75;
  }

  private async validateAudioTranscription(englishName: string, arabicName: string) {
    try {
      // تحويل النص إلى صوت والتحقق
      return true;
    } catch (error) {
      return false;
    }
  }

```

```

    }
}

private getPhonetics(text: string): string {
    return text.toLowerCase().replace(/[^a-z0-9]/g, '');
}

private calculateSimilarity(str1: string, str2: string): number {
    const longer = str1.length > str2.length ? str1 : str2;
    const shorter = str1.length > str2.length ? str2 : str1;

    if (longer.length === 0) return 1.0;

    const editDistance = this.getEditDistance(longer, shorter);
    return (longer.length - editDistance) / longer.length;
}

private getEditDistance(s1: string, s2: string): number {
    const costs = [];
    for (let i = 0; i ≤ s1.length; i++) {
        let lastValue = i;
        for (let j = 0; j ≤ s2.length; j++) {
            if (i === 0) {
                costs[j] = j;
            } else if (j > 0) {
                let newValue = costs[j - 1];
                if (s1.charAt(i - 1) !== s2.charAt(j - 1)) {
                    newValue = Math.min(Math.min(newValue, lastValue), costs[j]) + 1;
                }
                costs[j - 1] = lastValue;
                lastValue = newValue;
            }
        }
        if (i > 0) costs[s2.length] = lastValue;
    }
    return costs[s2.length];
}
}

```

التدخل اليدوي

التدخل اليدوي يتم فقط لـ

- قبول الإضافة - إدراج السجل في قاعدة البيانات ✓
- رفض الإضافة - حذف السجل المعلق ✕
- تعديل البيانات - تصحيح أو تحديث المعلومات ✎

```

// server/manual-review.ts
export interface ReviewAction {
  recordId: string;
  action: 'accept' | 'reject' | 'modify';
  reason?: string;
  modifications?: Partial<SanctionRecord>;
  reviewedBy: string;
}

export async function reviewBotSubmission(action: ReviewAction) {
  const record = await db.query.pendingRecords.findOne({
    where: { id: action.recordId }
  });

  if (action.action === 'accept') {
    // قبول الإضافة
    await db.insertSanctionRecord(record);
    console.log(`✓ تم قبول : ${record.nameEn}`);
  } else if (action.action === 'reject') {
    // رفض الإضافة
    await db.query.pendingRecords.delete({
      where: { id: action.recordId }
    });
    console.log(`✗ تم رفض : ${record.nameEn}`);
  } else if (action.action === 'modify') {
    // تعديل البيانات
    await db.query.pendingRecords.update(
      { id: action.recordId },
      action.modifications
    );
    console.log(`✎ تم تعديل : ${record.nameEn}`);
  }

  // تسجيل العملية
  await db.query.auditLog.insert({
    action: action.action,
    recordId: action.recordId,
    reason: action.reason,
    modifications: action.modifications,
    reviewedBy: action.reviewedBy,
    timestamp: new Date()
  });
}

```



```
});  
}
```

الخلاصة

البنية التحتية تجمع بين

- ✓ **تقنيات حديثة** - React 19, Node.js, TypeScript
- ✓ **أداء عالي** - Redis, Indexes, Caching
- ✓ **أمان شامل** - Encryption, Authentication
- ✓ **أتمتة ذكية** - Smart Bot للتقائمية
- ✓ **مراجعة يدوية** - للتحكم والجودة

النظام جاهز للعمل مع الجهات المختصة والمؤسسات المالية.